Rule of Five, Destructors, and Object Slicing - Advanced Concepts

Theory, Edge Cases & Code Examples

- ✓ Complex Interactions and Compiler Generation
- ✓ Polymorphic Destruction Safety
- ✓ The Silent Polymorphism Killer
- ✓ Deleting Derived Objects Through Base Pointers Without Virtu
- ✓ Pure Virtual Destructors Require Definitions

→ *Swipe to learn* →

Rule of Five, Destructors, and Object Slicing - Advanced Concepts

Theory

📌 1. Rule of Five - Complex Interactions and Compiler Generation (1/3)

How declaring one special member affects others:

You Declare	Default Ctor	Destructor	Copy Ctor	Copy Assign	Move Ctor	Move Assign
Nothing	✅ Generated	✅ Generated	✅ Generated	✅ Generated	✅ Generated	✅ Generated
Destructor	✅ Generated	User-defined	✅ Generated (deprecated)	✅ Generated (deprecated)	❌ **Suppressed**	❌ **Suppressed**
Copy Constructor	✅ Generated	✅ Generated	User-defined	✅ Generated	❌ **Suppressed**	❌ **Suppressed**
Copy Assignment	✅ Generated	✅ Generated	✅ Generated	User-defined	❌ **Suppressed**	❌ **Suppressed**
Move Constructor	✅ Generated	✅ Generated	❌ **Suppressed**	❌ **Suppressed**	User-defined	✅ Generated
Move Assignment	✅ Generated	✅ Generated	❌ **Suppressed**	❌ **Suppressed**	✅ Generated	User-defined

CRITICAL: Declaring any special member triggers suppression of others. Modern practice: **explicitly define or delete all five** when any one is user-defined.

Rule of Five, Destructors, and Object Slicing - Advanced Concepts

Theory

📌 1. Rule of Five - Complex Interactions and Compiler Generation (2/3)

Rule of Five + Virtual Destructors in inheritance:

Base Class Configuration	Effect on Derived	Recommended Pattern
Virtual destructor only	Derived moves suppressed	Explicitly default moves in base
Virtual + defaulted all five	Derived can generate all	✅ **Best practice**
Virtual + user-defined copies	Derived copies work, moves suppressed	Define moves too
Non-virtual destructor	❌ **Dangerous**	Makes polymorphism unsafe

Rule of Five, Destructors, and Object Slicing - Advanced Concepts

Theory

📌 1. Rule of Five - Complex Interactions and Compiler Generation (3/3)

Code example - proper Rule of Five with virtual destructor:

```
class PolymorphicBase {
protected:
    int* data;
public:
    PolymorphicBase() : data(new int[100]) {}

    // ✅ Virtual destructor
    virtual ~PolymorphicBase() { delete[] data; }

    // ✅ Explicitly define or default all five
    PolymorphicBase(const PolymorphicBase& other)
        : data(new int[100]) {
        std::copy(other.data, other.data + 100, data);
    }

    PolymorphicBase& operator=(const PolymorphicBase& other) {
        if (this != &other) {
            int* temp = new int[100];
            std::copy(other.data, other.data + 100, temp);
            delete[] data;
            data = temp;
        }
        return *this;
    }

    PolymorphicBase(PolymorphicBase&& other) noexcept
        : data(other.data) {
        other.data = nullptr;
    }

    PolymorphicBase& operator=(PolymorphicBase&& other) noexcept {
        if (this != &other) {
            delete[] data;
            data = other.data;
            other.data = nullptr;
        }
        return *this;
    }
};
```

Rule of Five, Destructors, and Object Slicing - Advanced Concepts

Theory

📌 2. Virtual Destructors - Polymorphic Destruction Safety (1/2)

Why virtual destructors are mandatory for polymorphic classes:

Scenario	Without Virtual Destructor	With Virtual Destructor
`delete basePtr;` (pointing to derived)	❌ **Only base destructor runs (UB)**	✅ Derived destructor → Base destructor
Resource cleanup	❌ **Derived resources leak**	✅ All resources cleaned up
Smart pointers (`unique_ptr`, `shared_ptr`)	❌ **Still need virtual!**	✅ Correct destruction
Vtable overhead	+8 bytes per object (vptr)	Same overhead (safety worth it)
Observable behavior	❌ Undefined behavior	✅ Well-defined

Common misconceptions:

Misconception	Reality
"Smart pointers don't need virtual destructors"	❌ FALSE - smart pointers still require virtual destructors in polymorphic hierarchies
"Virtual destructors are slow"	Vtable overhead is negligible (one pointer); safety benefit is enormous
"I don't delete through base pointers"	Code evolves; future maintainers may delete polymorphically
"Destructors don't need to be virtual if empty"	Virtual dispatch applies to *calling* the destructor, not *contents*

Rule of Five, Destructors, and Object Slicing - Advanced Concepts

Theory

📌 2. Virtual Destructors - Polymorphic Destruction Safety (2/2)

Code example - virtual destructor requirement:

```
class Base {
public:
    virtual ~Base() { std::cout << "~Base\n"; }  // ✅ Virtual
};

class Derived : public Base {
    int* buffer;
public:
    Derived() : buffer(new int[10000]) {}
    ~Derived() override {
        delete[] buffer;  // ✅ Called when deleting through Base*
        std::cout << "~Derived\n";
    }
};

// Usage:
Base* ptr = new Derived();
delete ptr;  // ✅ Calls ~Derived() then ~Base()
// Without virtual: only ~Base() runs → 10000 ints leaked!
```

Rule of Five, Destructors, and Object Slicing - Advanced Concepts

Theory

📌 3. Object Slicing - The Silent Polymorphism Killer (1/3)

What slicing does to objects:

Aspect	Original Derived Object	After Slicing to Base
Object size	sizeof(Derived)	sizeof(Base) - **smaller!**
Derived data members	Present	❌ **Lost/discarded**
Vtable pointer	Points to Derived vtable	❌ **Changed to Base vtable**
Virtual function calls	Calls Derived overrides	❌ **Calls Base implementations**
Type	Truly a Derived object	❌ **Truly a Base object**

When slicing occurs:

Context	Slicing?	Example
Pass by value to function	✅ YES	`void func(Base b)` called with Derived
Return by value (different type)	✅ YES	`Base func() { return Derived(); }`
Assignment to base	✅ YES	`Base b = derivedObj;`
Container insertion	✅ YES	`vector<Base> v; v.push_back(derivedObj);`
Pass by reference	❌ NO	`void func(Base& b)` - safe
Pass by pointer	❌ NO	`void func(Base* b)` - safe
Smart pointer containers	❌ NO	`vector<unique_ptr<Base>>` - safe

Rule of Five, Destructors, and Object Slicing - Advanced Concepts

Theory

📌 3. Object Slicing - The Silent Polymorphism Killer (2/3)

Prevention strategies:

Technique	Implementation	Effectiveness
Delete copy operations	`Base(const Base&) = delete;`	✅ Compile-time error
Protected copy operations	`protected: Base(const Base&);`	✅ Prevents external slicing
Pass by reference	`void func(const Base& obj)`	✅ Preserves polymorphism
Pass by pointer	`void func(Base* obj)`	✅ Explicit lifetime
Smart pointer containers	`vector<unique_ptr<Base>>`	✅ Container polymorphism
Virtual clone method	`virtual unique_ptr<Base> clone()`	✅ Explicit polymorphic copy

Rule of Five, Destructors, and Object Slicing - Advanced Concepts

Theory

📌 3. Object Slicing - The Silent Polymorphism Killer (3/3)

Code example - slicing in action:

```
class Animal {
public:
    int id = 0;
    virtual void speak() { std::cout << "Animal\n"; }
    virtual ~Animal() = default;
};

class Dog : public Animal {
public:
    double weight = 25.5;
    void speak() override { std::cout << "Woof! (weight: " << weight << ")\n"; }
};

// ❌ DANGER: Slicing function
void makeSound(Animal a) {  // Pass by value
    a.speak();  // Always prints "Animal" - polymorphism destroyed!
}

// ✅ SAFE: Polymorphic function
void makeSoundSafe(const Animal& a) {  // Pass by reference
    a.speak();  // Correctly calls Dog::speak() for Dog objects
}

int main() {
    Dog myDog;
    myDog.id = 42;
    myDog.weight = 30.0;

    makeSound(myDog);      // Output: "Animal" - sliced!
    makeSoundSafe(myDog);  // Output: "Woof! (weight: 30)" - correct!
}
```

Key takeaway: Virtual destructors are **mandatory** for polymorphic classes to prevent UB and resource leaks. Object slicing silently destroys polymorphism - always pass polymorphic objects by reference or pointer, never by value. Use deleted/protected copy operations to prevent slicing at compile time.

Rule of Five, Destructors, and Object Slicing - Advanced Concepts

Edge Cases

📌 1. Deleting Derived Objects Through Base Pointers Without Virtual Destructors

The most dangerous scenario in C++ inheritance: deleting a derived object through a base pointer when the base class lacks a virtual destructor causes undefined behavior. Only the base class destructor executes, leaking all resources allocated by the derived class and potentially corrupting memory.

```
class Base {
public:
    ~Base() {  // ❌ Not virtual
        std::cout << "Base destructor\n";
    }
};

class Derived : public Base {
    int* largeArray;
public:
    Derived() : largeArray(new int[10000]) {}
    ~Derived() {
        delete[] largeArray;  // ❌ Never called through base pointer
        std::cout << "Derived destructor\n";
    }
};

int main() {
    Base* ptr = new Derived();
    delete ptr;  // ❌ UB: only Base::~Base() runs, leaks largeArray
}
```

This undefined behavior is particularly insidious because it may appear to work in debug builds or simple test cases, only to cause crashes or memory leaks in production. Modern compilers may warn about this, but it's not guaranteed. The fix is simple but must be applied consistently: make base class destructors virtual whenever the class is intended for polymorphic use.

Rule of Five, Destructors, and Object Slicing - Advanced Concepts

Edge Cases

📌 2. Pure Virtual Destructors Require Definitions

A pure virtual destructor makes a class abstract while still allowing proper cleanup, but unlike other pure virtual functions, it **must** have a body defined. This is because destructors are always called during object destruction, even for pure virtual ones.

```
class AbstractBase {
public:
    virtual ~AbstractBase() = 0;  // ✅ Makes class abstract
};

// ❌ Linker error without this definition
AbstractBase::~AbstractBase() {
    std::cout << "AbstractBase cleanup\n";
}

class Concrete : public AbstractBase {
public:
    ~Concrete() override {
        std::cout << "Concrete cleanup\n";
    }
};

int main() {
    AbstractBase* obj = new Concrete();
    delete obj;  // Calls Concrete::~Concrete() then AbstractBase::~AbstractBase()
}
```

The pure virtual destructor declaration prevents instantiation of the base class while ensuring the derived destructor chain executes properly. Forgetting to define the pure virtual destructor body causes cryptic linker errors about undefined symbols, confusing developers unfamiliar with this requirement.

Rule of Five, Destructors, and Object Slicing - Advanced Concepts

Edge Cases

📌 3. Deleted Destructors for Lifetime Control

Explicitly deleting the destructor prevents object destruction, forcing controlled lifetime management through factory functions or specific deletion methods. This pattern enforces heap-only allocation or ensures objects are destroyed only through specific interfaces.

```
class ControlledLifetime {
private:
    ControlledLifetime() {}
    ~ControlledLifetime() = delete;  // ❌ Cannot destroy normally

public:
    static ControlledLifetime* create() {
        return new ControlledLifetime();
    }

    void destroy() {
        this->~ControlledLifetime();  // ✅ Can call from within class
        operator delete(this);
    }
};

int main() {
    // ControlledLifetime obj;  // ❌ Error: deleted destructor
    ControlledLifetime* obj = ControlledLifetime::create();
    // delete obj;  // ❌ Error: deleted destructor
    obj->destroy();  // ✅ OK: controlled destruction
}
```

This idiom is useful for reference-counted objects, objects managed by pools, or resources requiring specific cleanup sequences. The deleted destructor prevents accidental stack allocation or direct `delete` calls, forcing users through the controlled interface.

Rule of Five, Destructors, and Object Slicing - Advanced Concepts

Edge Cases

📌 4. Object Slicing During Function Parameter Passing

Object slicing silently occurs when passing derived objects by value to functions accepting base class parameters. The derived portion is discarded, and polymorphic behavior is lost because the vtable pointer is changed to point to the base class table.

```
class Shape {
public:
    int id = 0;
    virtual void draw() { std::cout << "Shape\n"; }
};

class Circle : public Shape {
public:
    double radius = 5.0;
    void draw() override { std::cout << "Circle with radius " << radius << "\n"; }
};

void render(Shape s) {  // ❌ Pass by value causes slicing
    s.draw();  // Always calls Shape::draw(), never Circle::draw()
}

int main() {
    Circle c;
    c.id = 42;
    render(c);  // Output: "Shape" (not "Circle")
}
```

The Circle object is sliced to a Shape object when passed to render(), losing the radius member and reverting draw() to Shape's implementation. This violates the principle of polymorphism and is almost never intentional. Always pass polymorphic objects by reference or pointer to preserve their full type.

Rule of Five, Destructors, and Object Slicing - Advanced Concepts

Edge Cases

📌 5. Slicing in Container Insertions

Standard containers store elements by value, causing object slicing when derived objects are inserted into containers of base class type. This destroys polymorphism and loses derived class data, making container-based polymorphism impossible with direct object storage.

```
std::vector<Shape> shapes;
Circle c;
c.radius = 10.0;
shapes.push_back(c);  // ❌ Slicing: stores only Shape part

shapes[0].draw();  // Prints "Shape", not "Circle"
```

The solution is to store pointers (raw or smart) to the base class instead of objects directly. Using `std::vector<std::unique_ptr<Shape>>` preserves polymorphism because the container stores pointers, not objects, avoiding slicing entirely.

Rule of Five, Destructors, and Object Slicing - Advanced Concepts

Edge Cases

📌 6. Smart Pointers and Virtual Destructors

Smart pointers like `std::shared_ptr` and `std::unique_ptr` don't eliminate the need for virtual destructors. While they manage deletion automatically, they determine the deleter based on the pointer type, requiring a virtual destructor for correct polymorphic cleanup.

```
class Base {
public:
    ~Base() { std::cout << "Base\n"; }  // ❌ Not virtual
};

class Derived : public Base {
public:
    ~Derived() { std::cout << "Derived\n"; }
};

int main() {
    std::shared_ptr<Base> ptr = std::make_shared<Derived>();
    // ❌ When ptr is destroyed, only Base::~Base() runs
}
```

The shared_ptr's deleter is set at construction to match the static type, but without a virtual destructor, the wrong destructor chain executes. Virtual destructors are essential for polymorphic smart pointer usage, a commonly misunderstood requirement.

Rule of Five, Destructors, and Object Slicing - Advanced Concepts

Code Examples

📌 1. Correct Virtual Destructor Implementation

This demonstrates proper virtual destructor usage in an inheritance hierarchy. The base class destructor is virtual, ensuring that deleting through a base pointer correctly calls the derived destructor first, then the base destructor, preventing resource leaks.

```
class Resource {
    int* data;
public:
    Resource(int size) : data(new int[size]) {
        std::cout << "Resource allocated\n";
    }

    virtual ~Resource() {  // ✅ Virtual for safe inheritance
        delete[] data;
        std::cout << "Resource freed\n";
    }
};

class TrackedResource : public Resource {
    std::string name;
public:
    TrackedResource(int size, std::string n)
        : Resource(size), name(n) {
        std::cout << "TrackedResource " << name << " created\n";
    }

    ~TrackedResource() override {
        std::cout << "TrackedResource " << name << " destroyed\n";
    }
};

int main() {
    Resource* r = new TrackedResource(100, "MyResource");
    delete r;  // ✅ Both destructors called in correct order
}
```

Rule of Five, Destructors, and Object Slicing - Advanced Concepts

Code Examples

📌 2. Pure Virtual Destructor Pattern

Pure virtual destructors provide a way to make a class abstract without requiring any pure virtual functions for operations. The destructor still needs a body because the destructor chain always executes, unlike pure virtual functions which may never be called.

```
class IInterface {
public:
    virtual void operation() = 0;
    virtual ~IInterface() = 0;  // ✅ Makes class abstract via destructor
};

// ✅ Must define, even though pure
IInterface::~IInterface() {
    std::cout << "IInterface cleanup\n";
}

class Implementation : public IInterface {
public:
    void operation() override {
        std::cout << "Implementation\n";
    }

    ~Implementation() override {
        std::cout << "Implementation cleanup\n";
    }
};

int main() {
    // IInterface i;  // ❌ Error: abstract class
    IInterface* i = new Implementation();
    i->operation();
    delete i;  // ✅ Both destructors execute
}
```

Rule of Five, Destructors, and Object Slicing - Advanced Concepts

Code Examples

📌 3. Deleted Destructor for Factory Pattern

The deleted destructor enforces that connections can only be closed through the disconnect() method, preventing accidental destruction and ensuring proper cleanup sequences. This pattern is useful for resources requiring specific shutdown procedures.

```
class DatabaseConnection {
private:
    std::string connectionString;

    DatabaseConnection(std::string conn)
        : connectionString(conn) {}

    ~DatabaseConnection() = delete;  // ❌ Prevents direct destruction

public:
    static DatabaseConnection* connect(std::string conn) {
        return new DatabaseConnection(conn);
    }

    void disconnect() {
        std::cout << "Disconnecting from " << connectionString << "\n";
        this->~DatabaseConnection();  // ✅ Accessible internally
        operator delete(this);
    }
};

int main() {
    DatabaseConnection* db = DatabaseConnection::connect("localhost:5432");
    // delete db;  // ❌ Error: deleted destructor
    db->disconnect();  // ✅ Controlled cleanup
}
```

Rule of Five, Destructors, and Object Slicing - Advanced Concepts

Code Examples

📌 4. Preventing Slicing Through Interface Design

Making copy operations protected in the base class prevents slicing at compile time. Users must pass polymorphic objects by reference or pointer, and cloning must be explicit through a virtual clone() method that preserves the full derived type.

```
class Shape {
public:
    virtual void draw() const = 0;
    virtual std::unique_ptr<Shape> clone() const = 0;
    virtual ~Shape() = default;

protected:
    // ✅ Protected copy operations prevent slicing
    Shape(const Shape&) = default;
    Shape& operator=(const Shape&) = default;
};

class Circle : public Shape {
    double radius;
public:
    explicit Circle(double r) : radius(r) {}

    void draw() const override {
        std::cout << "Circle with radius " << radius << "\n";
    }

    std::unique_ptr<Shape> clone() const override {
        return std::make_unique<Circle>(*this);
    }
};

void render(const Shape& s) {  // ✅ Pass by reference
    s.draw();
}

int main() {
    Circle c(10.0);
    render(c);  // ✅ Polymorphism works, no slicing

    // Shape s = c;  // ❌ Error: copy constructor is protected
}
```

Rule of Five, Destructors, and Object Slicing - Advanced Concepts

Code Examples

📌 5. Safe Container Usage with Polymorphism

Storing smart pointers to the base class in containers preserves polymorphism. Deleting copy operations in the base class prevents accidental slicing while clarifying that these objects should only exist via pointers or references.

```
class Animal {
public:
    virtual void speak() const = 0;
    virtual ~Animal() = default;

protected:
    Animal() = default;
    Animal(const Animal&) = delete;  // ✅ Prevent slicing
    Animal& operator=(const Animal&) = delete;
};

class Dog : public Animal {
public:
    void speak() const override {
        std::cout << "Woof!\n";
    }
};

class Cat : public Animal {
public:
    void speak() const override {
        std::cout << "Meow!\n";
    }
};

int main() {
    // std::vector<Animal> animals;  // ❌ Would slice

    std::vector<std::unique_ptr<Animal>> animals;  // ✅ Stores pointers
    animals.push_back(std::make_unique<Dog>());
    animals.push_back(std::make_unique<Cat>());

    for (const auto& animal : animals) {
        animal->speak();  // ✅ Polymorphism preserved
    }
}
```

Rule of Five, Destructors, and Object Slicing - Advanced Concepts

Code Examples

📌 6. Trivial vs Non-Trivial Destructors

Trivial destructors enable optimizations like memcpy for bulk operations and affect whether a type is POD (Plain Old Data). Even an empty user-defined destructor makes the type non-trivial, affecting move semantics and standard library optimizations.

```
struct Trivial {
    int x;
    // ✅ Implicitly generated trivial destructor
};

struct NonTrivial {
    int x;
    ~NonTrivial() {}  // ❌ User-defined, now non-trivial
};

static_assert(std::is_trivially_destructible_v<Trivial>);
static_assert(!std::is_trivially_destructible_v<NonTrivial>);

// Impact on memcpy safety and optimizations
Trivial arr1[100];
Trivial arr2[100];
std::memcpy(arr2, arr1, sizeof(arr1));  // ✅ Safe for trivial types

NonTrivial arr3[100];
NonTrivial arr4[100];
// std::memcpy(arr4, arr3, sizeof(arr3));  // ⚠️ Unsafe, skips destructors
std::copy(std::begin(arr3), std::end(arr3), std::begin(arr4));  // ✅ Safe
```

Rule of Five, Destructors, and Object Slicing - Advanced Concepts

Code Examples

📌 7. Move Operations Suppressed by Destructor

Defining a destructor suppresses automatic move constructor generation, forcing either inefficient copying or explicit move operation definition. This is a common source of performance bugs when legacy code adds destructors without considering move semantics.

```
class Resource {
public:
    Resource() = default;
    ~Resource() {}  // ❌ User-defined destructor

    // Move constructor NOT generated automatically
};

static_assert(!std::is_move_constructible_v<Resource>);

class BetterResource {
public:
    BetterResource() = default;
    ~BetterResource() {}

    // ✅ Explicitly default move operations
    BetterResource(BetterResource&&) = default;
    BetterResource& operator=(BetterResource&&) = default;
};

static_assert(std::is_move_constructible_v<BetterResource>);
```

Rule of Five, Destructors, and Object Slicing - Advanced Concepts

Code Examples

📌 8. Virtual Destructor Overhead Analysis

Virtual destructors add a vptr to each object, increasing size by one pointer (8 bytes on 64-bit systems). For classes with few data members, this overhead is significant, but for polymorphic classes managing resources, the safety benefit far outweighs the cost.

```
struct NoVirtual {
    int data;
};

struct WithVirtual {
    int data;
    virtual ~WithVirtual() = default;
};

int main() {
    std::cout << "NoVirtual: " << sizeof(NoVirtual) << " bytes\n";
    std::cout << "WithVirtual: " << sizeof(WithVirtual) << " bytes\n";
    // Typically: 4 vs 16 bytes (64-bit system with vptr)
}
```

What You Learned

Key Takeaways

- ✓ Complex Interactions and Compiler Generation
- ✓ Polymorphic Destruction Safety
- ✓ The Silent Polymorphism Killer
- ✓ Deleting Derived Objects Through Base Pointers Without Virtu

→ *Swipe to learn* →